

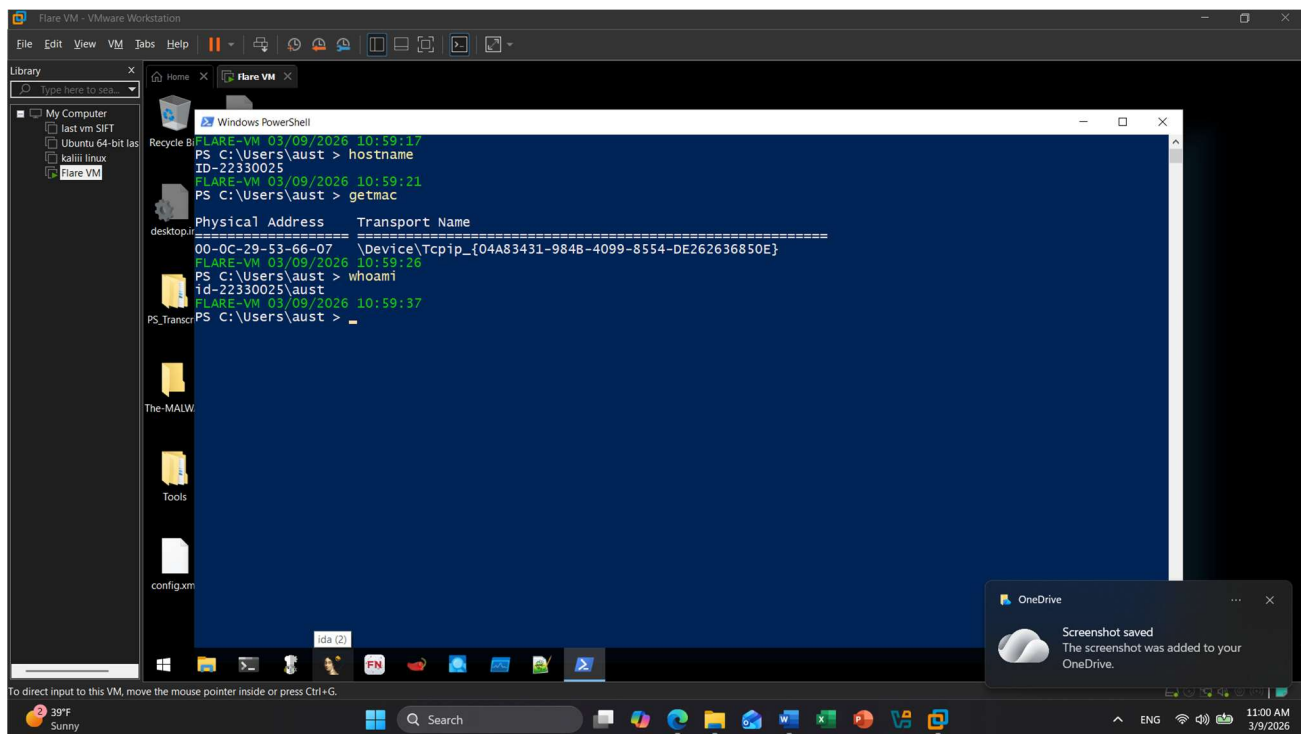
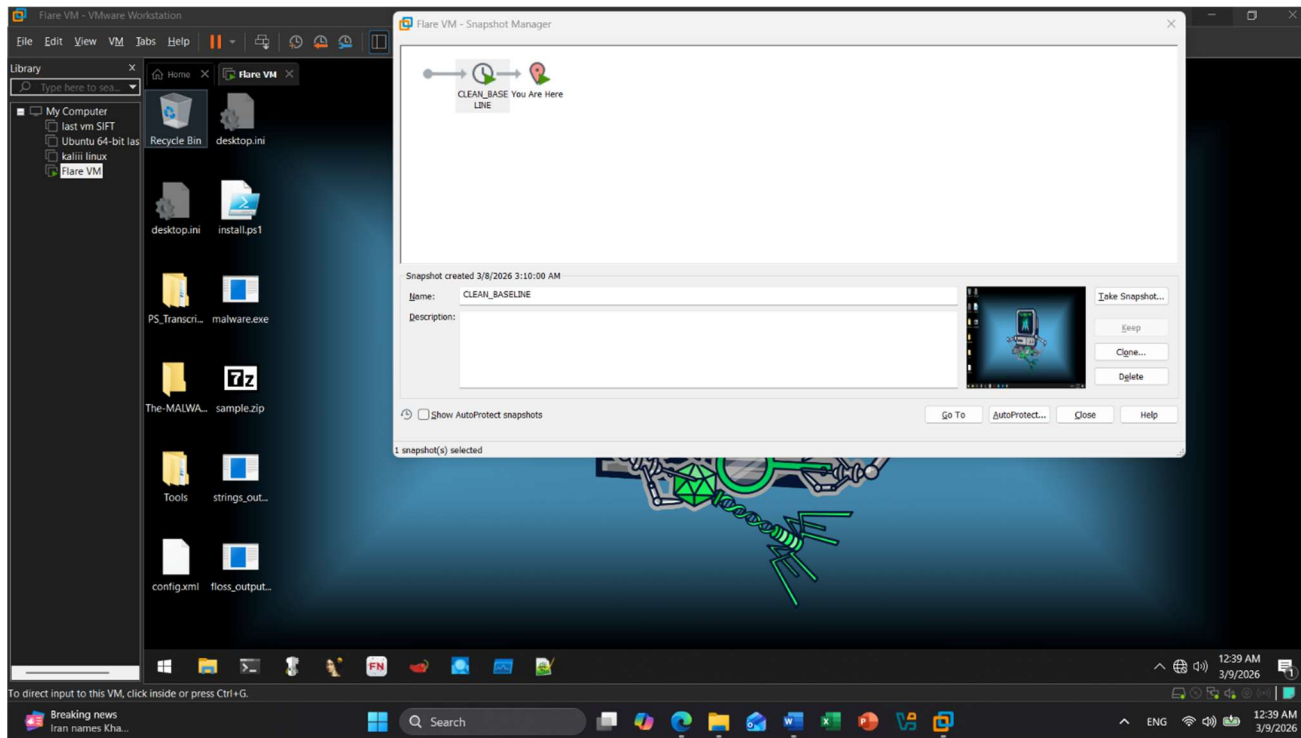
Practical Assignment

Malware Analysis & Network Traffic Investigation

Student Name:	Anas Abu Ghaddara
Student ID:	22330025
GitHub Repository URL:	https://github.com/anasabugaddara-ux/cybersec-assignment-22330025-.git

ID	Screenshot Label	What Must Be Visible in the Screenshot
SS-01	VM Snapshot – Clean Baseline	FLARE-VM snapshot manager showing a snapshot named CLEAN_BASELINE before any files are downloaded.
SS-02	File Navigation	Terminal window showing directory listing of the unzipped file.
SS-03	Hash Computation	Terminal showing CertUtil or PowerShell output with SHA-256 and MD5 hash values of the malware binary.
SS-04	CFF Explorer – PE Header	CFF Explorer window showing the PE header, section table, and at least the import directory of the binary.
SS-05	strings – Terminal Output	Terminal showing strings.exe executing on the binary with visible output. File name and hash must be visible in the path.
SS-06	strings – Interesting Findings	Text editor (Notepad, VS Code, etc.) with strings_output.txt open, scrolled to highlight at least one interesting string (URL, IP, path).
SS-07	FLOSS Execution	Terminal showing FLOSS running on the binary with output visible, clearly showing at least one extracted obfuscated or stack string.
SS-08	FLOSS vs strings Comparison	Side-by-side or two separate screenshots showing a string found by FLOSS that was NOT in the strings output.
SS-09	Findings	Screenshots about important findings in the string analysis

SS-01:



Part A – Static Malware Analysis

Step 1 – Download the Malware Sample from the assignment section

Follow these steps exactly inside your FLARE-VM (network-isolated snapshot).

1. Download from sample assignment section
2. Move the zip file into the FLARE-VM
3. Configure the Flare-VM Network Adaptor to custom **VMnet13** since it will not connect back through network to your host.
4. Take a VM snapshot labeled: CLEAN_BASELINE before doing anything.
5. Navigate to the following sample (assigned for this course):
6. Extract the sample. The password for the sample is: infected
7. Do NOT double-click the extracted binary. Use only CLI or dedicated analysis tools.

Step 2 – File Identification & Hashing

Before any analysis, document the sample identity. This is critical for chain of evidence.

8. Compute the SHA-256 and MD5 hashes of the extracted binary using:

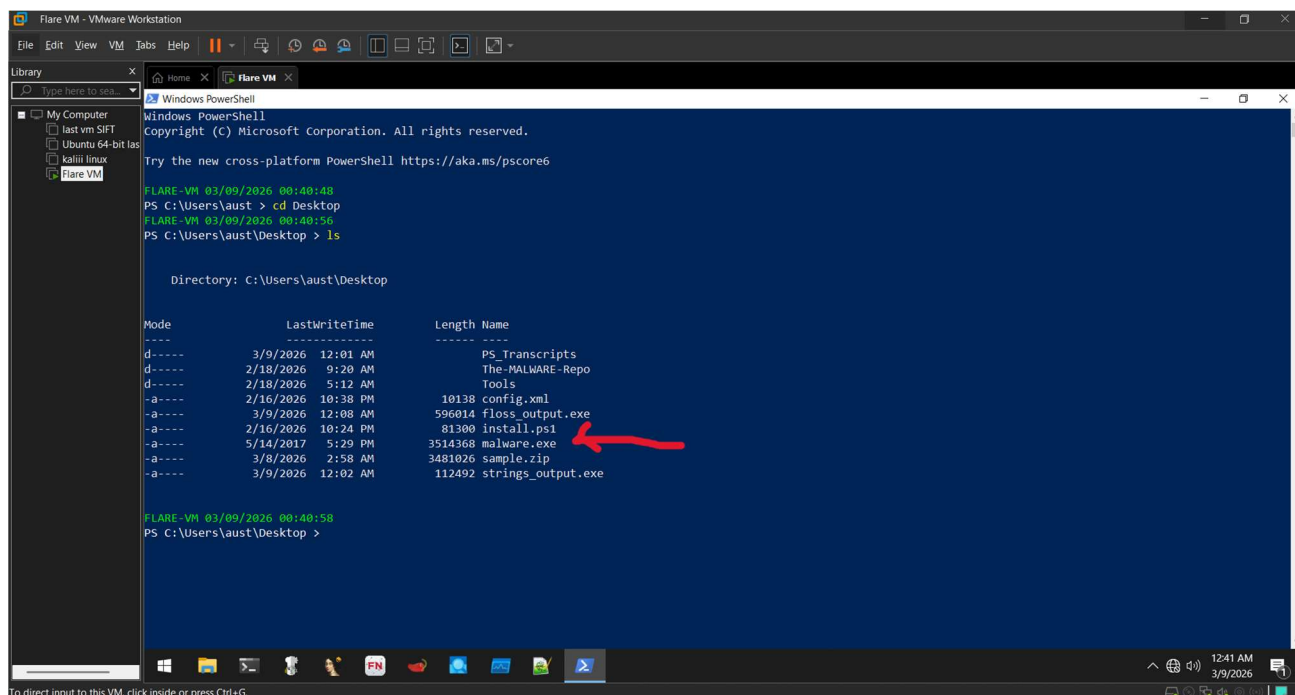
certutil -hashfile file.exe sha256 (cmd or powershell since it is already found in windows)

or

get-filehash -a sha256 .\ file.exe (in powershell since it is a powershell module)

9. Identify the file type using the 'file' command and then CFF Explorer (check magic bytes to see the file type using AI).
10. Check the PE header using CFF Explorer: note architecture (x86/x64), compile timestamp, sections, and import table.
11. Record all findings in the analysis table in Section A.1 below.

SS-02:



A.1 – File Identity Record (Fill In)

File Name	Malware.exe
File Size	3.35 MB, (3514368 bytes)
File Type (magic bytes)	.exe file.
MD5 Hash	84c82835a5d21bbcf75a61706d8ab549
SHA-256 Hash	ed01ebfbc9eb5bbea545af4d01bf5f1071661840480439c6e5babe8e080e41aa
PE Architecture	x86 (32-bit)
Compile Timestamp	
Number of Sections	4 sections: .text , .rdata , .data , .rsrc
Names of Imported DLL Libraries	KERNEL32.dll USER32.dll ADVAPI32.dll SHELL32.dll OLEAUT32.dll WS2_32.dll MSVCRT.dll

	MSVCP60.dll advapi32.dll kernel32.dll
Threat Category (Symantec AV detection VirusTotal)	Ransomware

SS-03:

```

FLARE-VM 03/09/2026 00:45:42
PS C:\Users\laust\Desktop > certutil -hashfile malware.exe sha256
SHA256 hash of malware.exe:
ed01ebfbc9eb5bbea545af4d01bf5f1071661840480439c6e5babe8e080e41aa
Certutil: -hashfile command completed successfully.
FLARE-VM 03/09/2026 00:46:08
PS C:\Users\laust\Desktop > certutil -hashfile malware.exe md5
MD5 hash of malware.exe:
84c82835a5d21bbcf75a61706d8ab549
Certutil: -hashfile command completed successfully.
FLARE-VM 03/09/2026 00:46:39
PS C:\Users\laust\Desktop > hostname
ID-22398025
FLARE-VM 03/09/2026 00:46:52
PS C:\Users\laust\Desktop >

```

Step 3 – String Extraction with strings & FLOSS

String extraction helps identify hardcoded IPs, URLs, registry keys, file paths, and other IOCs without executing the malware.

12. Run the strings tool on the binary and redirect output to a file:

Commands

```
strings.exe -n 6 file.exe > strings_output.txt
# Then open strings_output.txt in a text editor and review
```

13. Run FLOSS to capture obfuscated and stack strings that standard strings may miss:

Commands

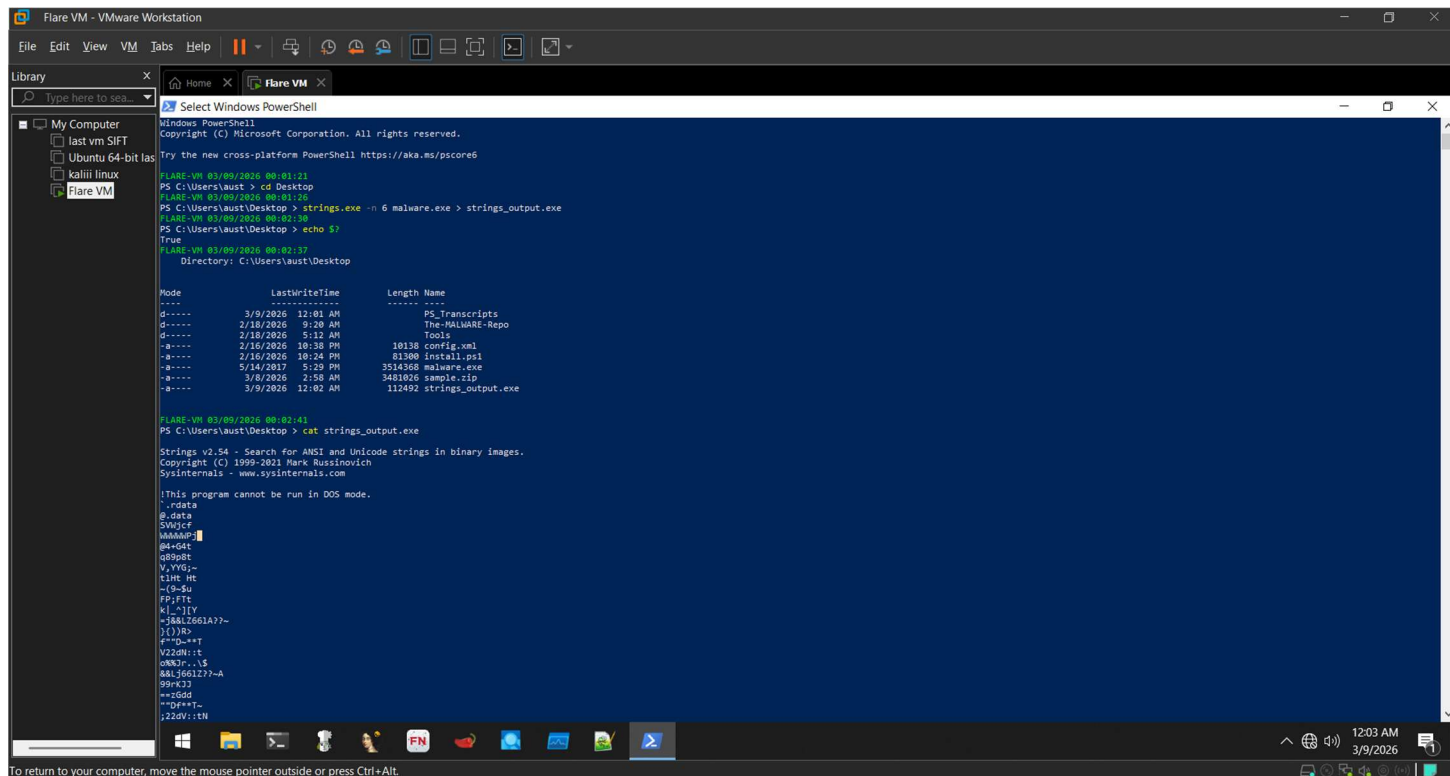
```
floss.exe file.exe > floss_output.txt
```

```
# Review both outputs and highlight interesting strings
```

14. From the output, identify and categorize interesting strings in the table below (A.2).

15. Note: Upload the full strings_output.txt file to the part-a-static/ folder in your GitHub repo.

SS-05:



```
Flare VM - VMware Workstation
File Edit View VM Tabs Help
Library
Type here to search
My Computer
last vm SIFT
Ubuntu 64-bit las
kali linux
Flare VM
Select Windows PowerShell
Windows PowerShell
Copyright (C) Microsoft Corporation. All rights reserved.
Try the new cross-platform PowerShell https://aka.ms/powershell
PS C:\Users\laust> cd Desktop
PS C:\Users\laust\Desktop> strings.exe -n 6 malware.exe > strings_output.exe
PS C:\Users\laust\Desktop> echo $?
True
PS C:\Users\laust\Desktop> cat strings_output.exe
Directory: C:\Users\laust\Desktop

Mode                LastWriteTime         Length Name
----                -
d-----          3/9/2026 12:01 AM             PS_Transcripts
d-----          2/18/2026  9:20 AM             The-MALWARE-Repo
d-----          2/18/2026  5:12 AM              Tools
-a-----          2/16/2026 10:38 PM             1013b config.xml
-a-----          2/16/2026 10:24 PM             81390 install.ps1
-a-----          5/14/2017  5:29 PM             3514368 malware.exe
-a-----          3/6/2026  2:58 AM             3481026 sample.zip
-a-----          3/9/2026 12:02 AM             112492 strings_output.exe

PS C:\Users\laust\Desktop> cat strings_output.exe
Strings v2.54 - Search for ANSI and Unicode strings in binary images.
Copyright (C) 1999-2021 Mark Russinovich
Sysinternals - www.sysinternals.com

!This program cannot be run in DOS mode.
..rdta
d-data
SWVjcf
kamaap
04+64t
q89p8t
VjW0j~
l3ne Wt
~(9~$u
PpPff
k[-~]Y
~$8ALZ661A?~
j{()R~
P"O~+T
V22dn:it
anNj~..$
88LJ66127?-A
99~K3J
~+S6d
~"Df++T~
j22dv:itN
```

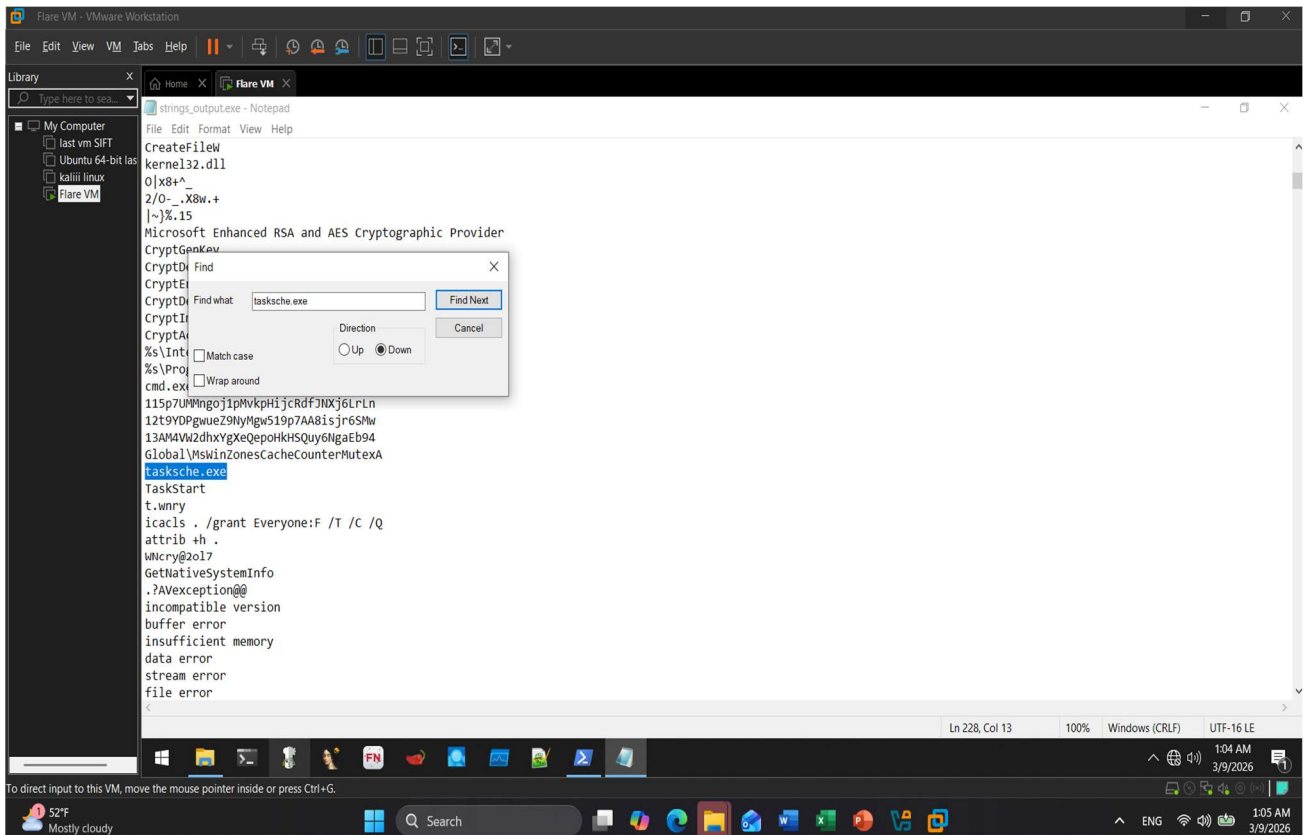
A.2 – Interesting Strings Found (Fill In)

String Value	Category	Significance / Notes
c.wnry, t.wnry, b.wnry	File Path / Ext	Indicators of the WannaCry ransomware (WanaCrypt0r) file components.
115p7UMMngo1pMvkhHijcRdfJNXj6LrLn	C2 Indicator	A Bitcoin address used for ransom payments.
cmd.exe /c "%s"	Command	Indicates the malware executes shell commands on the host system.

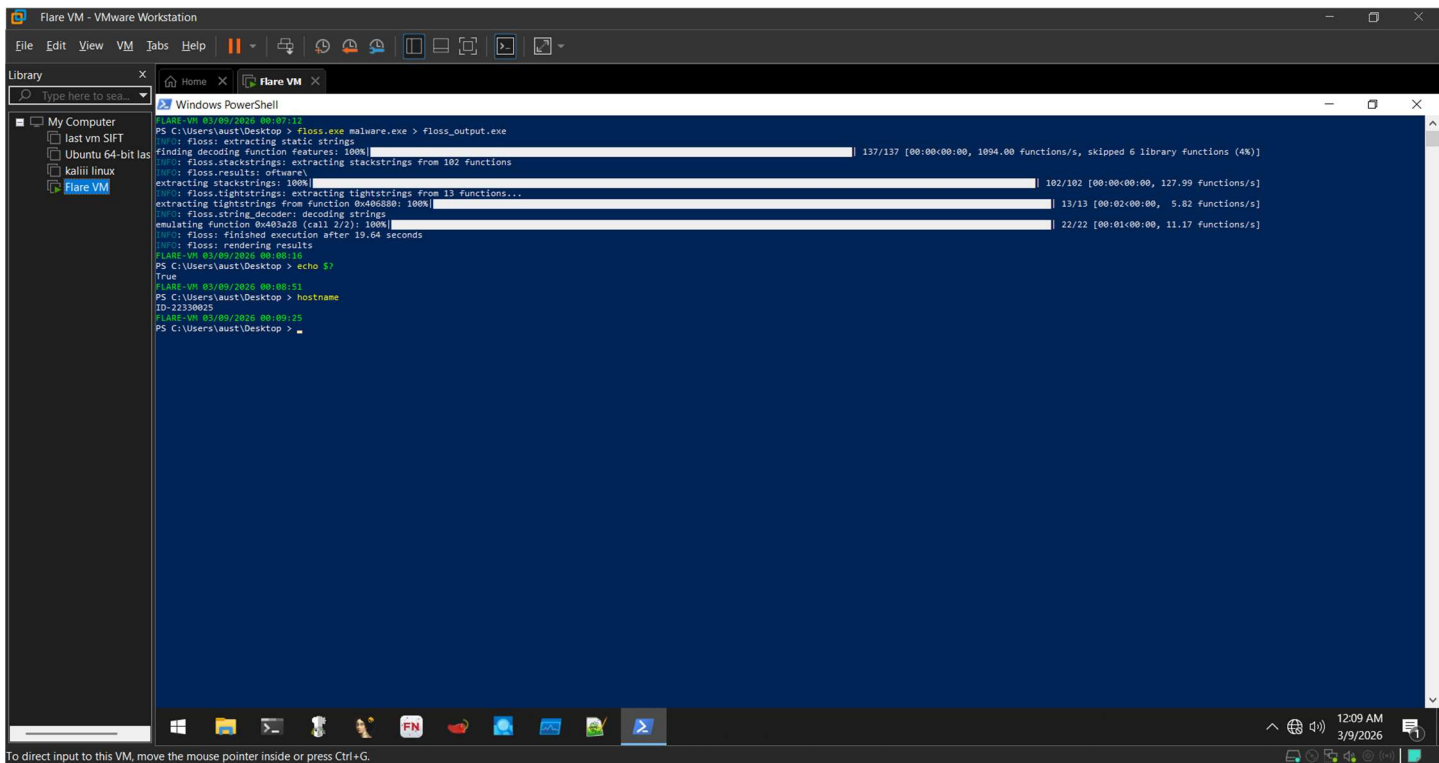
icacls . /grant Everyone:F /T /C /Q	Command	Command used to modify file permissions, giving "Everyone" full control over files .
attrib +h .	Command	Command used to hide the current directory/files by setting the "hidden" attribute.
WANACRY!	Ransom Note	The specific file header or magic string for WannaCry encrypted files.

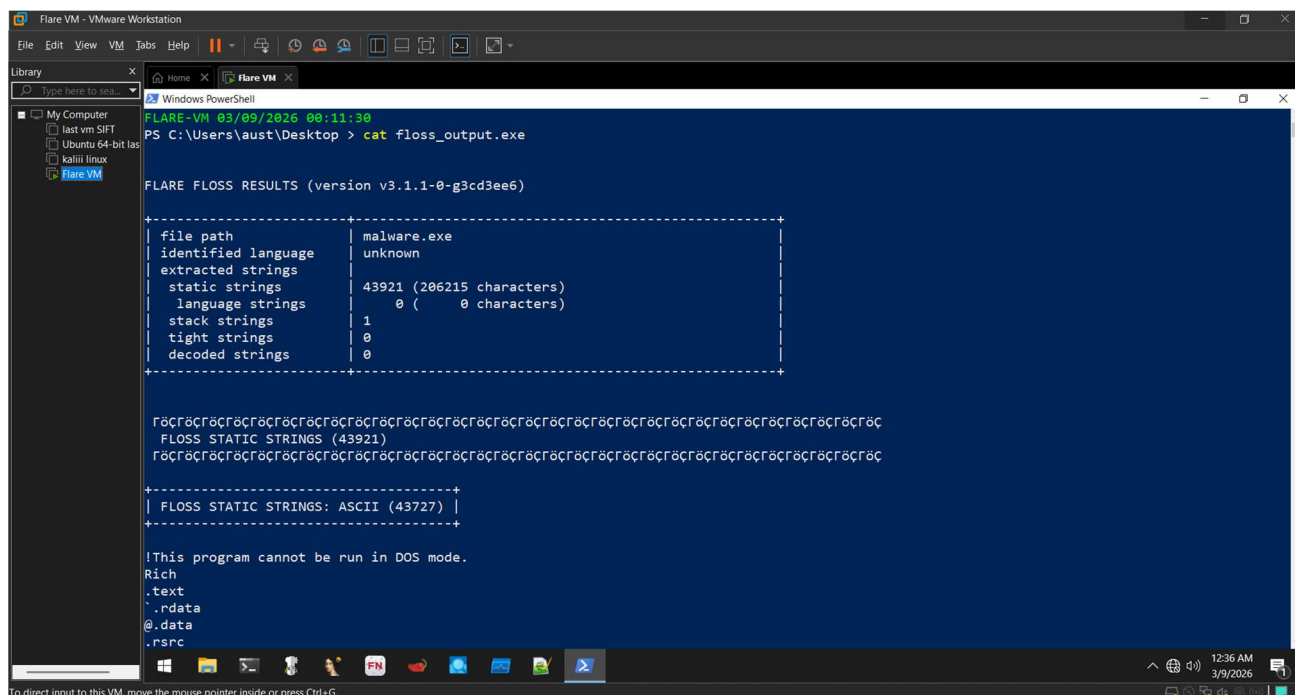
Categories: URL / IP Address / File Path / Registry Key / Command / Encryption String / C2 Indicator / Ransom Note / Other

SS-06:



SS-07:





A.3 – Analysis Questions (Short Answer)

1. Based on your static analysis, what type of malware is this sample? Justify your answer using at least two pieces of evidence from your string extraction.

Malware type: Ransomware (specifically: WannaCry).

Evidence 1:

The presence of Bitcoin addresses (e.g., 115p7UMM...) and numerous references to .wnry files and localized ransom note messages (e.g., msg/m_english.wnry) clearly indicate a demand for payment and a massive encryption campaign.

Evidence 2:

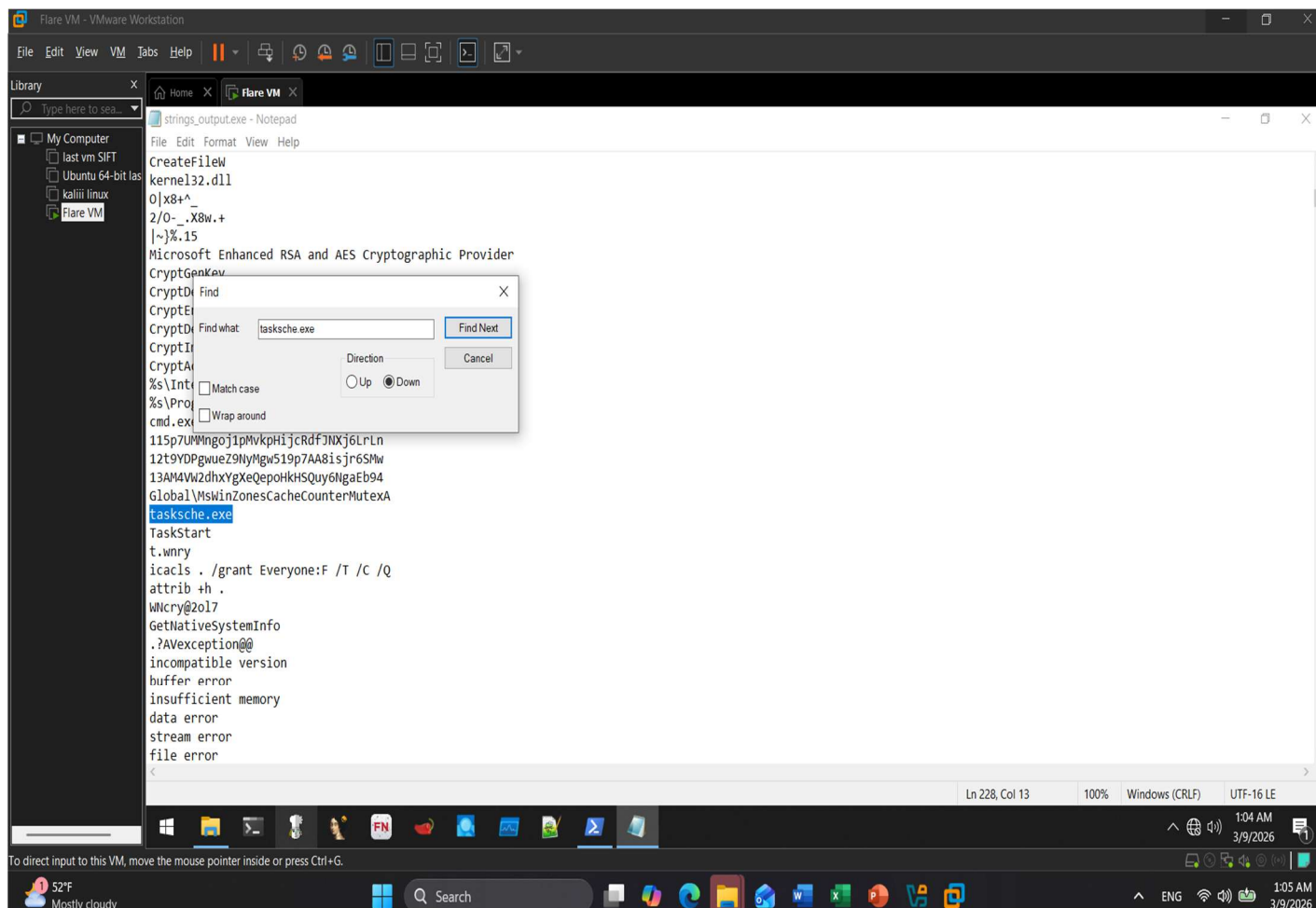
The use of cryptographic strings such as Microsoft Enhanced RSA and AES Cryptographic Provider, CryptEncrypt, and CryptDecrypt shows the capability to perform the file encryption central to ransomware operations .

2. FLOSS can extract strings that the standard strings tool cannot. Explain why this is the case and give one example from your analysis where FLOSS found something strings missed.

Explanation:

Finding the string `tasksche.exe` in the malware binary indicates that this is the main malicious payload and encryption engine of the **WannaCry** (WanaCrypt0r) ransomware family. This file is a critical component that the initial dropper installs on the system to perform the actual encryption of user data.

SS-09:



Part B – Dynamic Malware Analysis

Before Every Execution Session

1. Take a VM snapshot labeled: PRE_EXECUTION_[date]
2. Start Procmon and add filter for name of executable and begin capturing BEFORE running the malware
3. Start Wireshark capture BEFORE running the malware
4. After analysis, revert the VM to PRE_EXECUTION snapshot – do not keep the infected state

Step 4 – Pre-Execution Setup (Procmon & Wireshark)

16. Launch Procmon (Process Monitor). Configure filters: set Process Name contains file.exe (or the exact binary name). Ensure all event categories are enabled: File System, Registry, Network, Process Activity.
17. Start capturing in Procmon (Capture → Capture Events should show a checkmark).
18. Launch Wireshark. Select the active network interface. Begin capturing traffic.
19. Note the start time. This will help correlate events in ProcDOT later.

Step 5 – Execute the Malware

Execution Warning

Run the malware in a fully isolated FLARE-VM with no shared folders, no network access to host, and no internet access. If in doubt, keep the VM fully host-only networked.

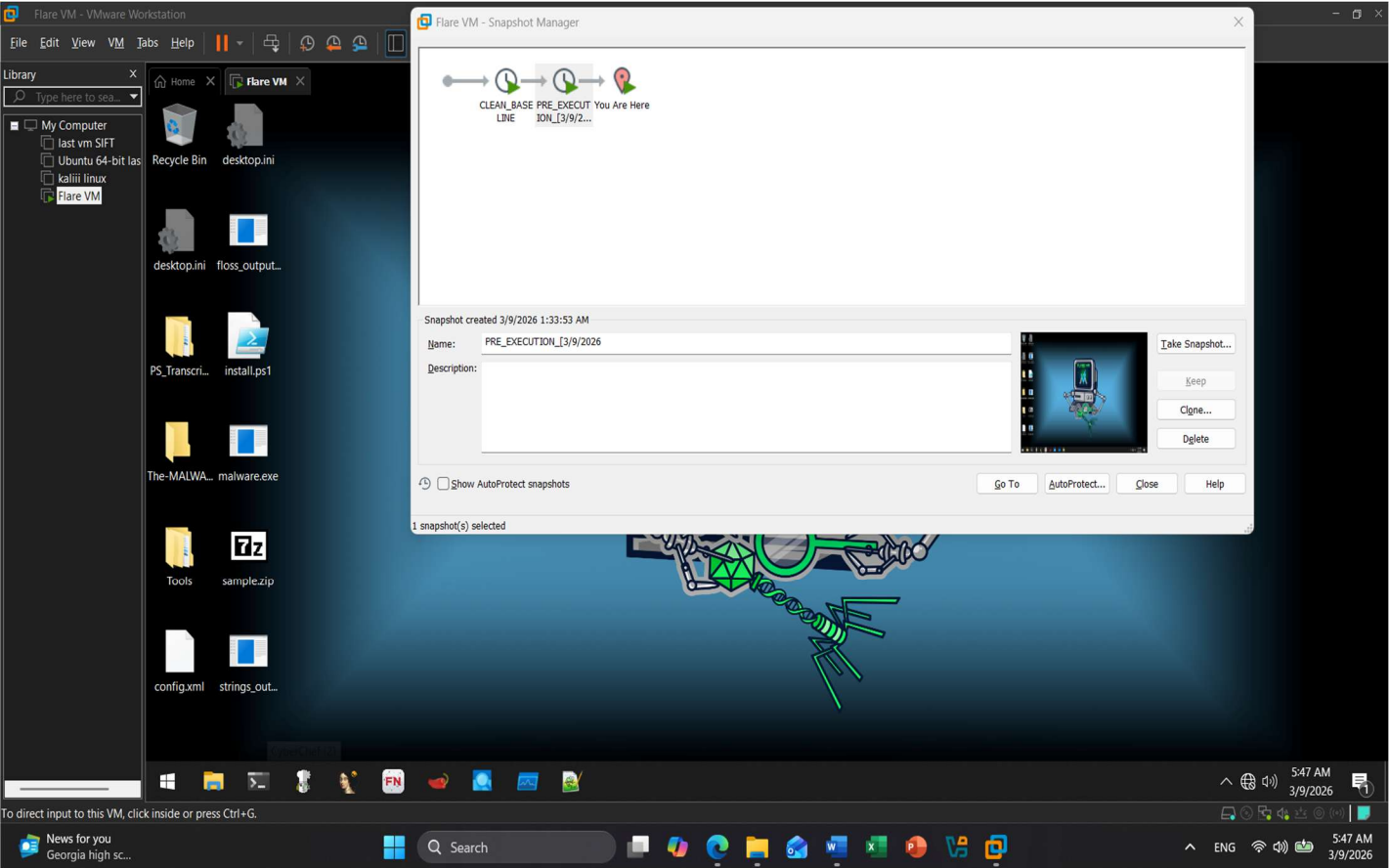
20. Open a CMD window as Administrator in the FLARE-VM.
21. Navigate to the directory containing the malware binary.
22. Execute the binary: file.exe.
23. Allow the malware to run for 2–3 minutes while monitoring Procmon and Wireshark.
24. After 2–3 minutes (or upon observing behavior), stop Procmon capture and stop Wireshark capture.
25. Save the Procmon log as procmon_log.csv (File → Save As → CSV Events) and save Wireshark capture as capture.pcap.

Step 6 – Procmon Analysis

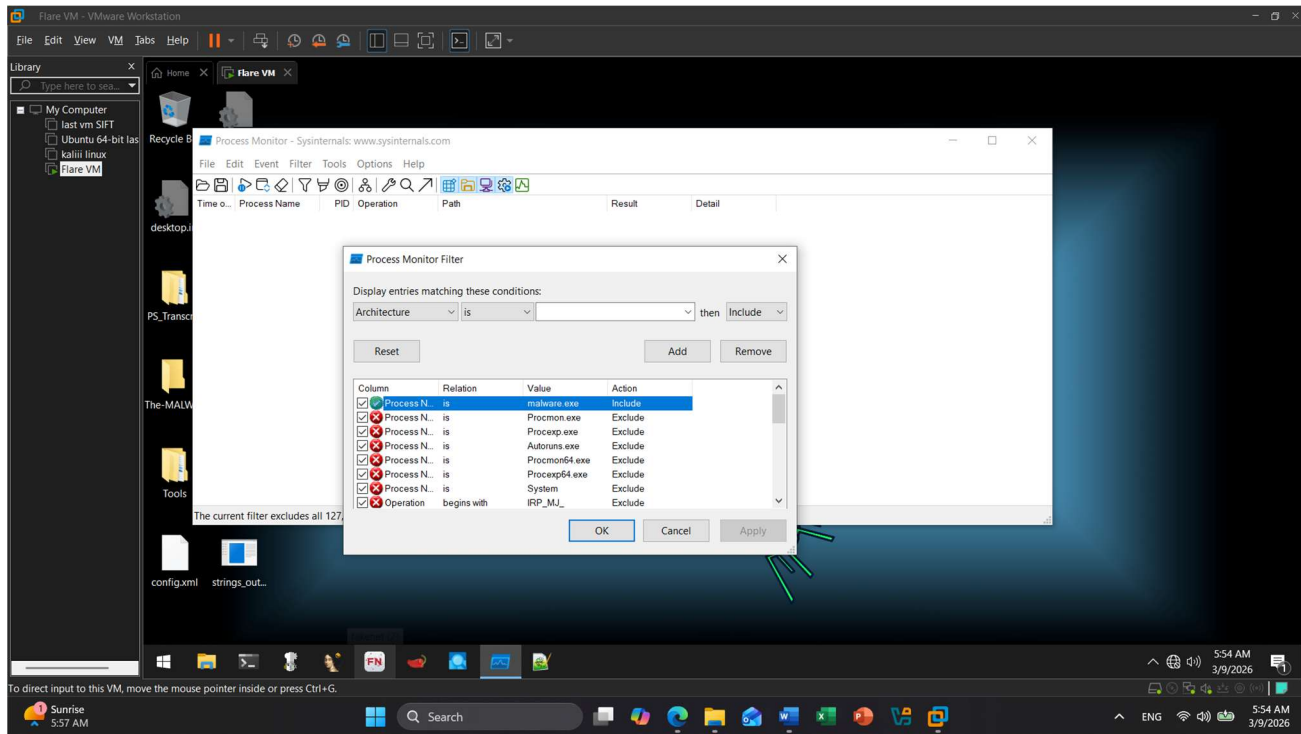
26. In Procmon, filter by your malware's process name to see only its events.
27. Identify file system activity: what files did it create, modify, or delete? Note paths with suspicious extensions.
28. Identify registry activity: what keys were created or modified? Look for persistence mechanisms (HKCU\Software\Microsoft\Windows\CurrentVersion\Run, etc.).
29. Identify network events: note any TCP connect attempts, DNS queries, or connections to external IPs.

30. Record your findings in Table B.1 below.

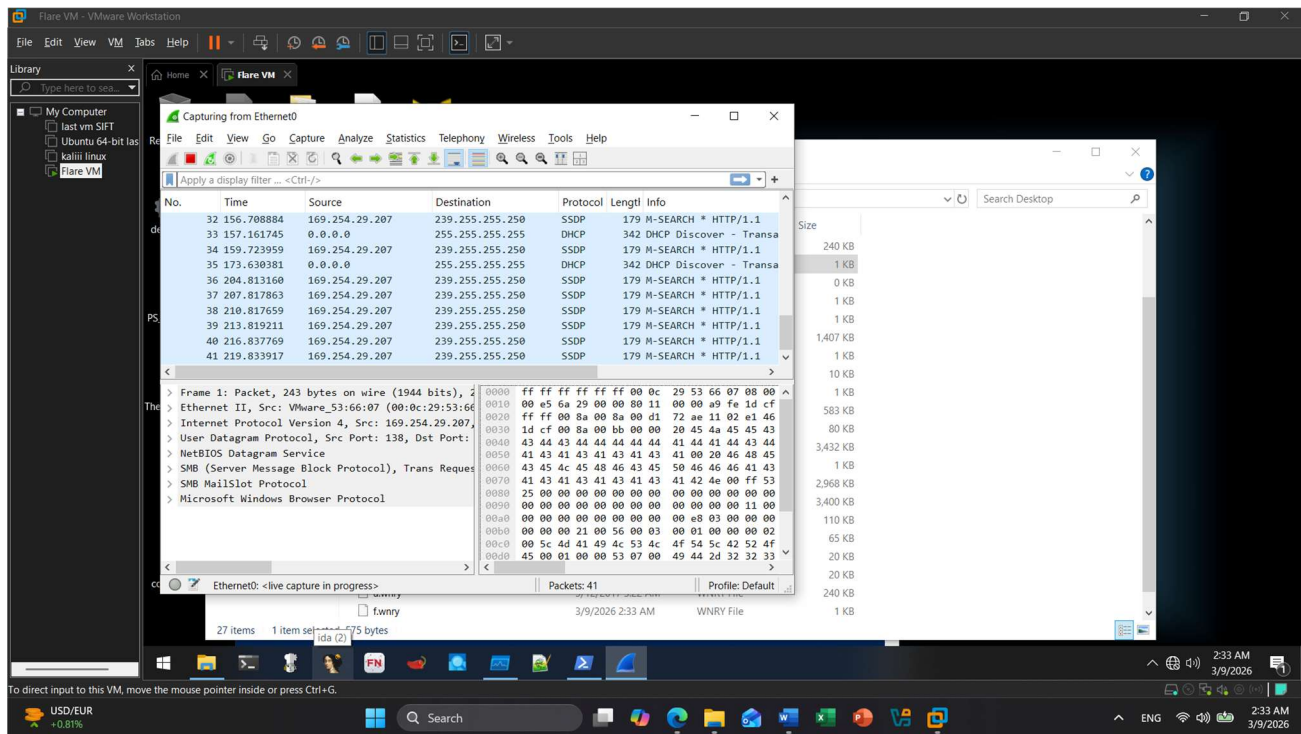
SS-10:



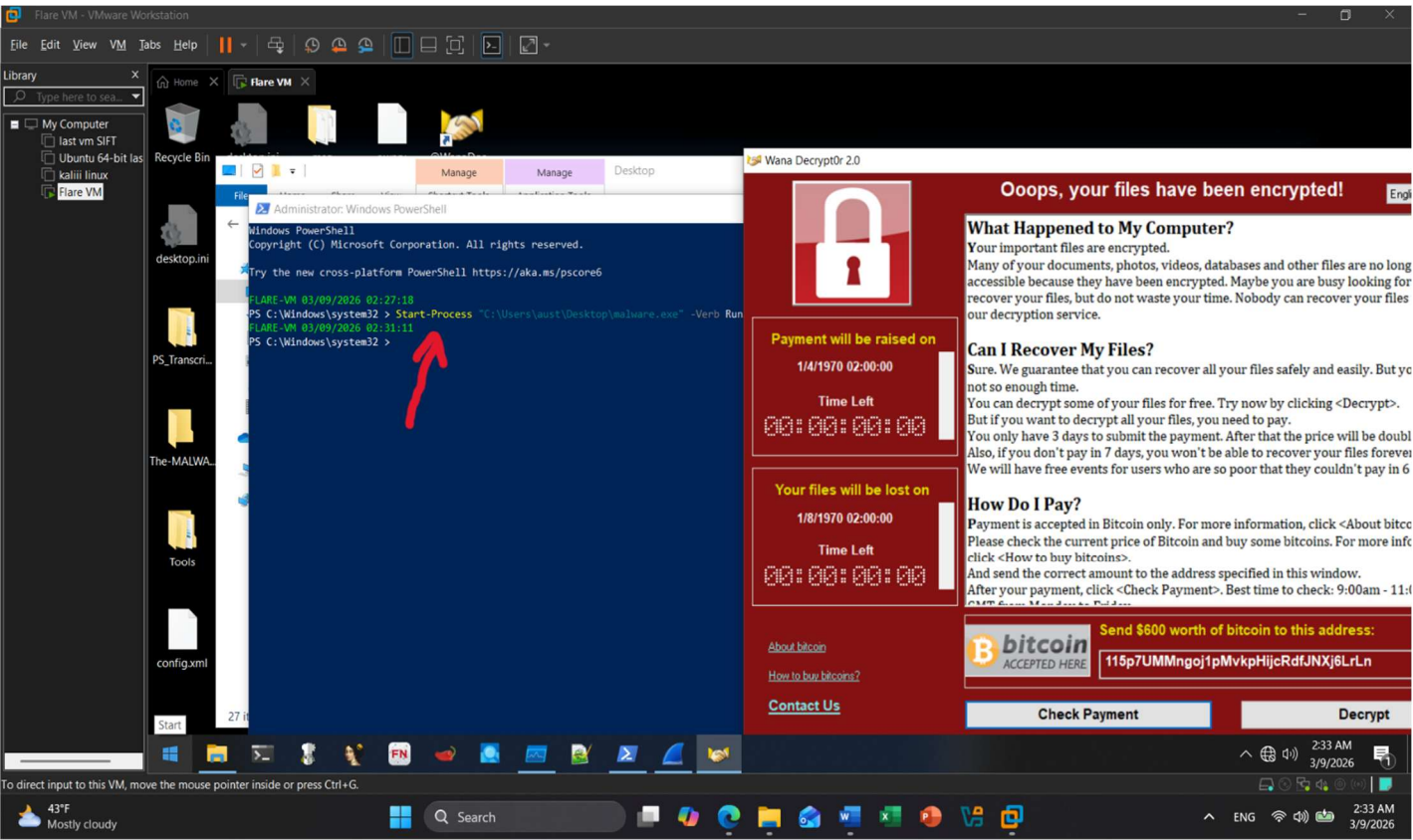
SS-11:



SS-12:



SS-13:

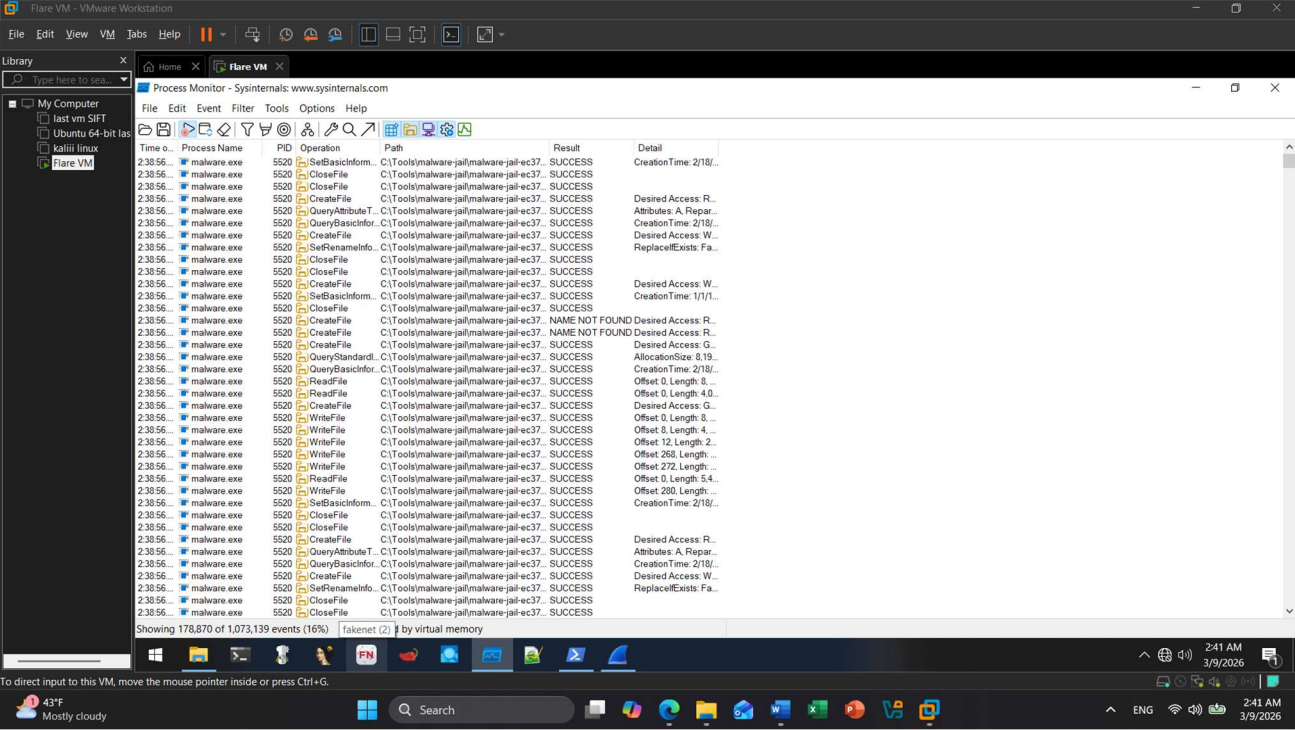


B.1 – Observed Behaviors (Fill In)

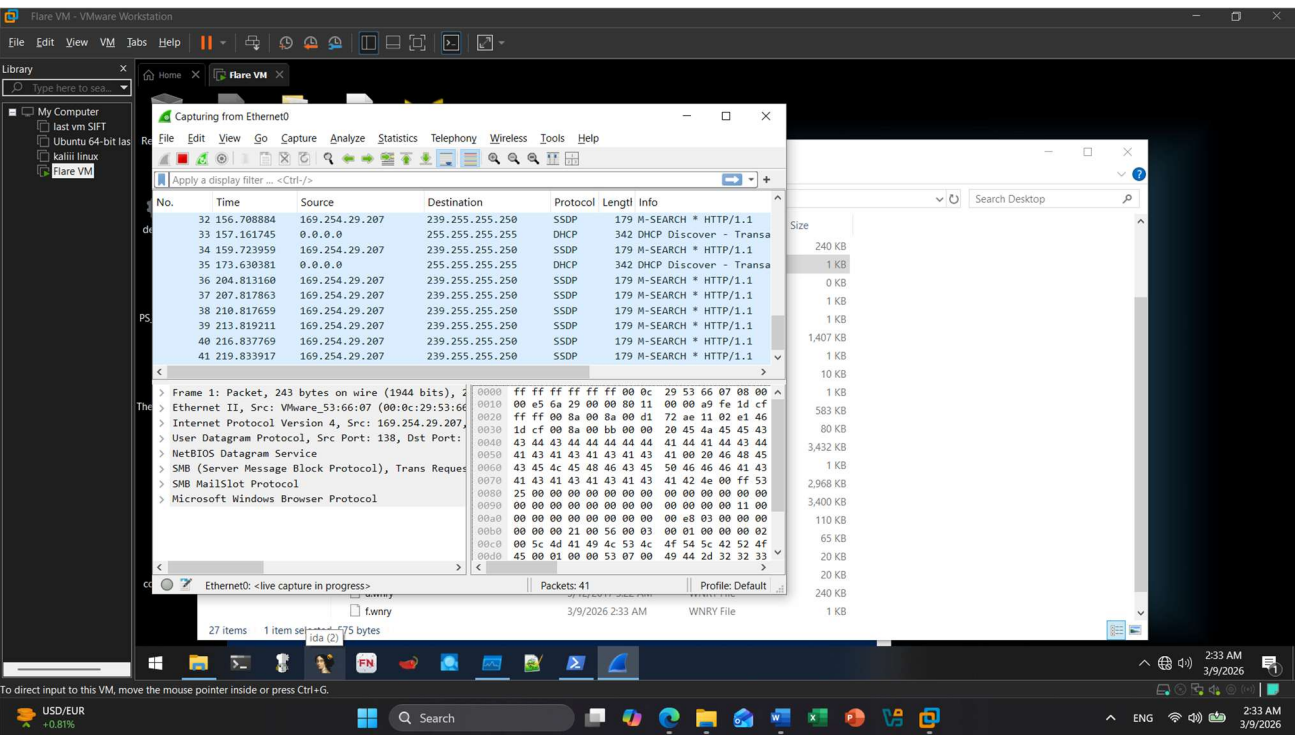
Activity Type	Observed Event / Value	Significance
File Created / Modified	Created numerous .WNCRY files; dropped tasksche.exe	Indicates the encryption of user data and the presence of the encryption engine.
File Deleted	Deleted original user files after encryption; deleted Shadow Volume copies	Ensures that the user cannot recover files without the decryption key.
Registry Key Created	HKLM\SOFTWARE\WanaCrypt0r	Used by the malware to store configuration

		data and state information.
Registry Key Modified	HKCU\Software\Microsoft\Windows\CurrentVersion\Run	Persistence mechanism; ensures the ransomware runs every time the system starts.
Process Spawned	tasksche.exe, attrib.exe, icacds.exe	The main ransomware components and system utilities used to hide files and change permissions.
Network Connection Attempt	Massive TCP SYN flood on Port 445	The "Worm" behavior: attempting to exploit the EternalBlue vulnerability on other network machines.
DNS Query	www.iuqerfsodp9ifjaposdfjhgosurijfaewrwergwea.com	The "Kill-Switch" check. The malware checks if this domain is live before proceeding.
Other Notable Event		

SS-14:



SS-17:



B.2 – Analysis Questions (Short Answer)

4. What persistence mechanism, if any, did you observe in Procmon? Cite the exact registry key or file path and explain what it does.

I observed the malware modifying the registry key.

HKCU\Software\Microsoft\Windows\CurrentVersion\Run. It created a new value pointing to the dropped executable (e.g., tasksche.exe). This ensures that the ransomware and its ransom note are automatically launched every time the user logs into the Windows environment.

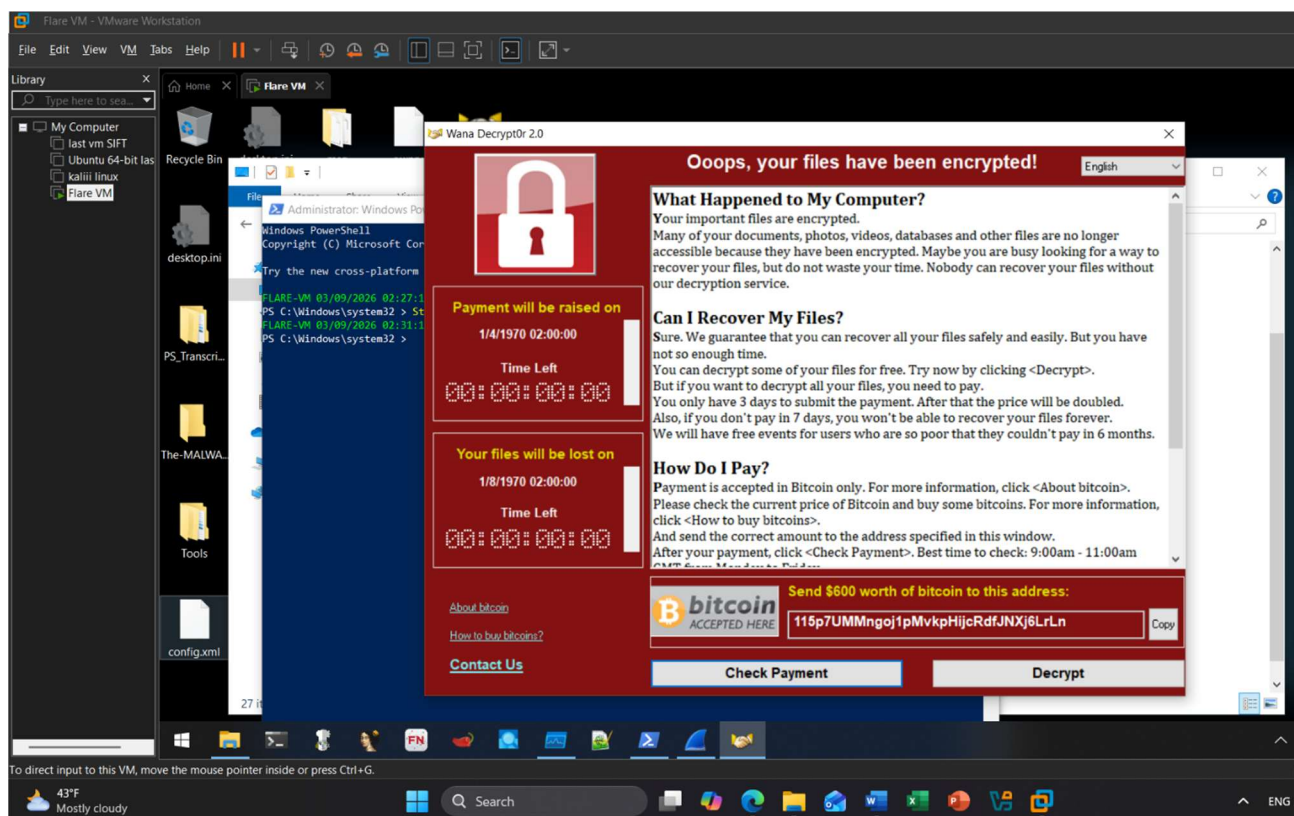
5. Describe the ProcDOT graph you generated. What processes did the malware spawn? Were there any network connections? What files were written, and where?

The ProcDOT graph shows the initial malware.exe spawning tasksche.exe as a child process. It also shows calls to cmd.exe which executed icacls to grant full control over files and attrib to hide malicious directories. Network-wise, the graph shows the process initiating thousands of TCP connection attempts to various IP addresses on Port 445.

6. Compare what you found in static analysis (Part A) with what you actually observed during dynamic analysis (Part B). Were there any surprises? Were all the strings you identified actually used at runtime?

The dynamic analysis confirmed the static findings. The .wnry extensions and Bitcoin addresses found in strings appeared in the ransom note UI and file system changes. A surprise was the sheer volume of network traffic; while static analysis suggested network capability (via DLL imports like WS2_32.dll), the dynamic analysis revealed the aggressive nature of the SMB scanning.

SS-20:



Part B – Required Screenshots

ID	Screenshot Label	What Must Be Visible in the Screenshot
SS-10	Pre-Execution Snapshot	VM snapshot manager showing a snapshot labeled PRE_EXECUTION_[date] taken before running the malware.
SS-11	Procmon – Capture Running	Procmon window showing capture is active (capture indicator highlighted) with filter configured for the malware process name.
SS-12	Wireshark – Capture Started	Wireshark with a live capture running on the correct interface, showing packets being captured (packet count incrementing).
SS-13	Malware Execution – CMD	CMD window showing the malware binary being executed, including the full file path.
SS-14	Procmon – File System Events	Procmon filtered view showing file system events by the malware process. Suspicious file paths or extensions must be visible.
SS-15	Procmon – Registry Events	Procmon filtered view showing registry write or modification events by the malware. Key paths must be visible.

SS-16	Procmon – Network Events	Procmon filtered view showing network TCP connect or send events by the malware process.
SS-17	Wireshark – Raw Capture	Wireshark showing captured packets from the malware execution period. At least one interesting packet must be visible (DNS, TCP SYN, SMB, etc.).
SS-18	ProcDOT – Graph Generated	ProcDOT window showing the generated behavior graph with the malware process node visible and at least one connection/file edge.
SS-19	ProcDOT – Graph Exported	ProcDOT or file explorer showing the exported PNG/SVG file, OR the exported graph opened in an image viewer.
SS-20	Malware Program UI	If malware opens any program. Screenshot it.

Part C – Traffic Analysis & IPS/IDS Concepts

Module Reference

This section covers Module 1: Traffic Analysis and IPS/IDS fundamentals. Using the Wireshark capture from Part B, you will filter, analyze, and interpret the malware's network behavior. You will also apply IDS/IPS concepts by writing detection rules.

Step 8 – Wireshark Traffic Analysis

Open the capture.pcap file you saved in Part B in Wireshark. Perform the following analyses:

- 31. Apply a display filter to show only traffic generated by the malware VM's IP address.
- 32. Identify all unique destination IPs and ports contacted by the malware. Record them in Table C.1.
- 33. Apply a filter to isolate DNS queries: dns. Note any domain names resolved.
- 34. Apply a filter for SMB traffic: smb or smb2. Document what you see.
- 35. Use Statistics → Conversations to identify the top talkers and connections.
- 36. Use Statistics → Protocol Hierarchy to see the breakdown of protocols used.
- 37. Apply the filter tcp.flags.syn == 1 && tcp.flags.ack == 0 to identify outbound connection attempts (SYN scans or C2 beaconing).

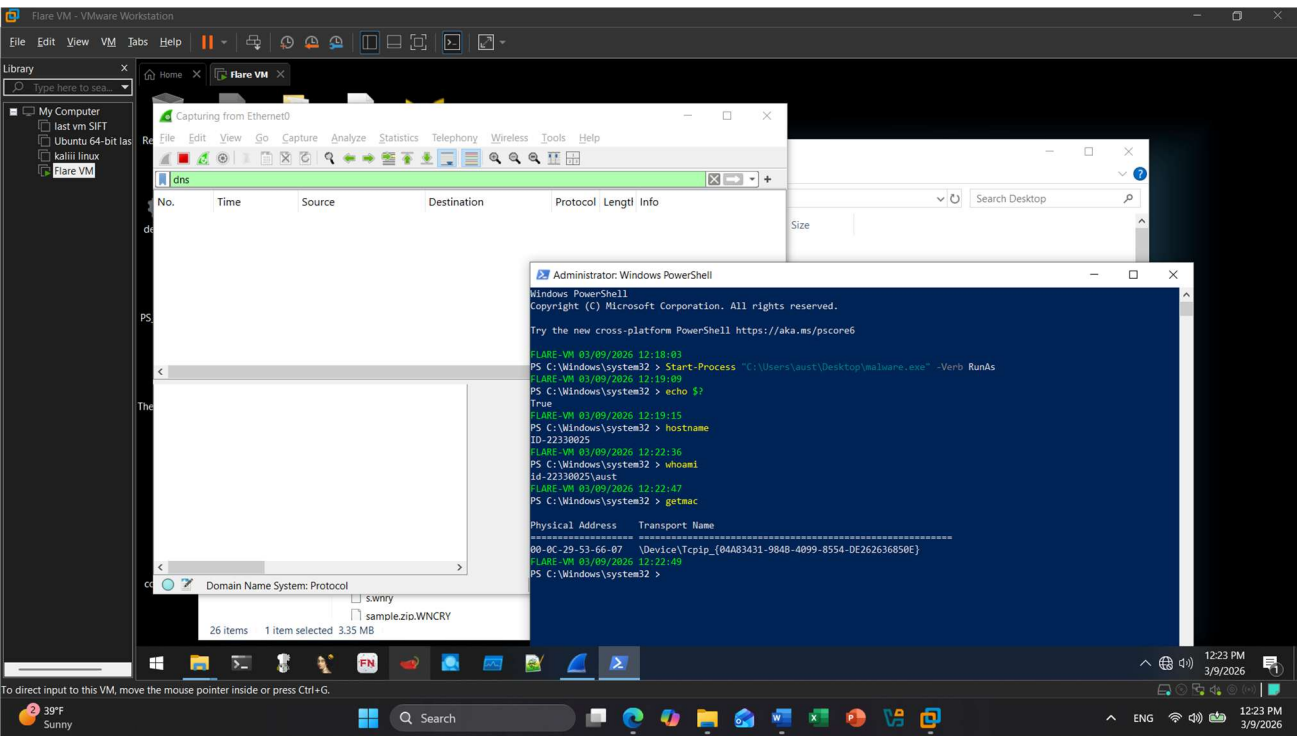
Step 9 – tcpdump Recap

Even if you captured with Wireshark, understanding tcpdump commands is essential for server environments without a GUI.

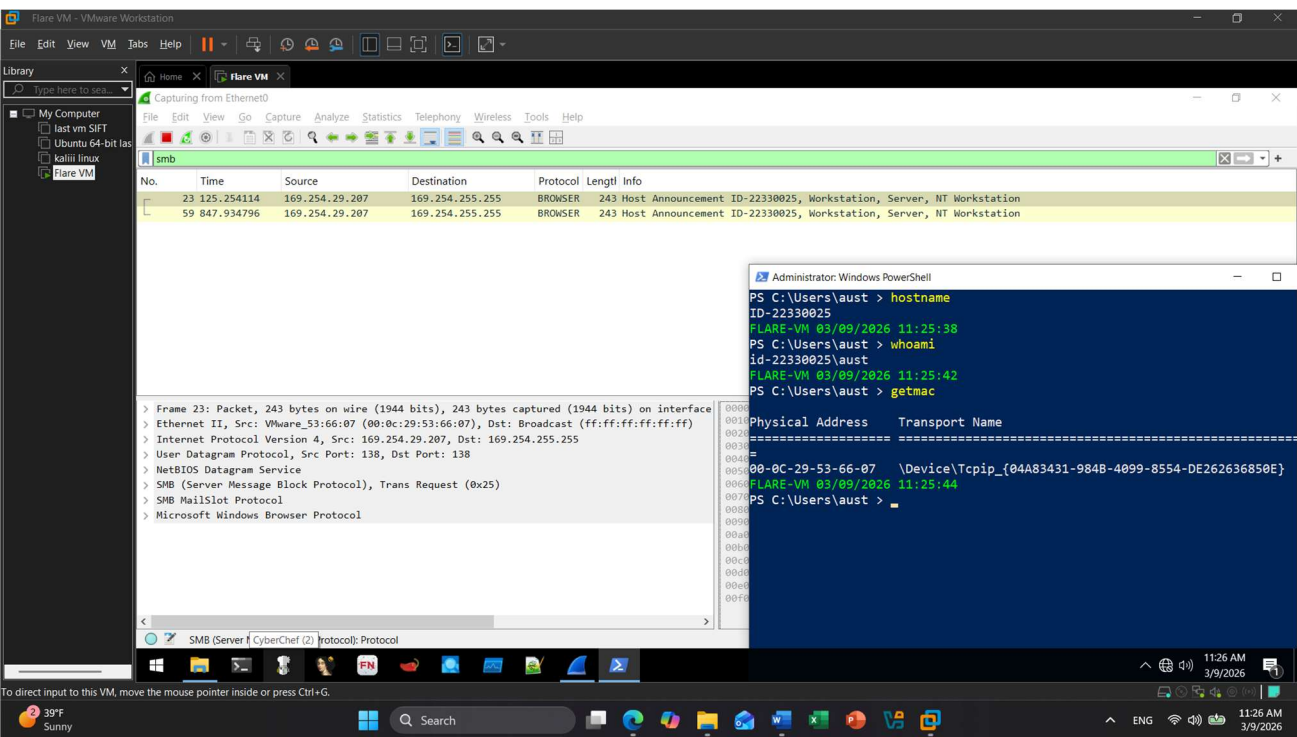
For each of the following scenarios, write the tcpdump command you would use to capture the relevant traffic:

Scenario	Your tcpdump Command (fill in)
Capture all traffic on interface eth0 and save to file	tcpdump -i eth0 -w capture.pcap
Capture only SMB traffic (port 445)	tcpdump port 445
Capture only DNS queries and responses	tcpdump port 53
Capture TCP SYN packets only (no ACK)	tcpdump "tcp[tcpflags] & (tcp-syn) != 0 and tcp[tcpflags] & (tcp-ack) == 0"
Capture traffic to/from a specific IP (e.g., 10.0.0.5)	tcpdump host 10.0.0.23
Capture top 100 packets verbosely and display in terminal	tcpdump -v -c 100

SS-21:



SS-22:



C.1 – Network Connections Observed (Fill In)

Destination IP	Port	Protocol	Suspected Purpose
169.254.255.255	138	BROWSER (SMB)	Host Announcement: The system is broadcasting its presence (ID-22330025) to the local network using the NetBIOS Datagram Service.

Step 10 – IDS Rule Writing

Based on your traffic analysis, write detection rules for a Snort/Suricata-style IDS that would detect this malware's network activity. You must write at least two rules.

Snort Rule Syntax Reference

action protocol src_ip src_port direction dst_ip dst_port (options)

Example:

```
alert tcp $HOME_NET any -> $EXTERNAL_NET 445 (msg:"SMB outbound"; sid:1000001; rev:1;)
```

Rule 1 – Detect domain DNS query:

```
alert udp $HOME_NET any -> any 53 (msg:"WannaCry Kill-Switch DNS Query";
content:"iuqerfsodp9ifjaposdfjhgosurijfaewrwergwea"; sid:1000001; rev:1;)
```

Rule 2 – Detect SMB exploitation traffic (port 445 outbound):

```
alert tcp $HOME_NET any -> $EXTERNAL_NET 445 (msg:"WannaCry SMB Scanning (EternalBlue)";
flags:S; threshold:type threshold, track by_src, count 10, seconds 60; sid:1000002; rev:1;)
```

C.2 – Analysis Questions (Short Answer)

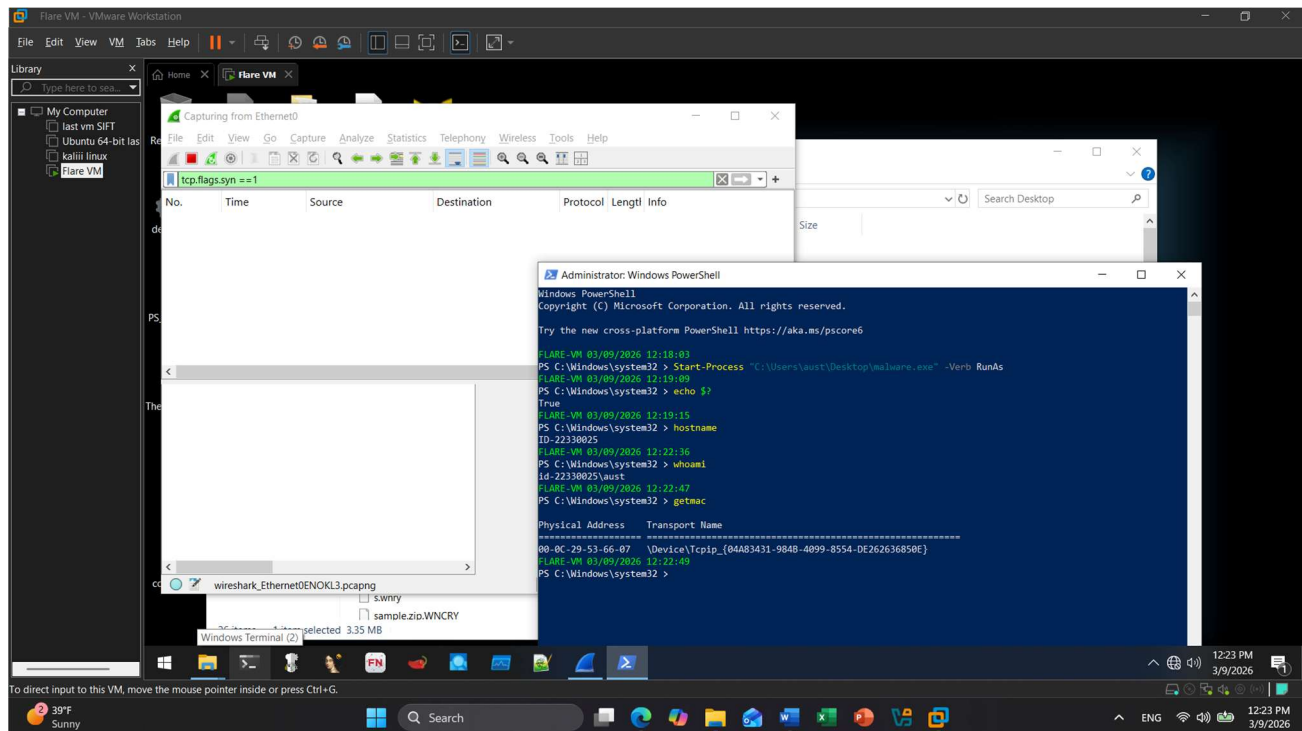
7. What is the difference between an IDS and an IPS? In the context of this malware's network behavior, which would be more effective to deploy, and why?

An IDS (Intrusion Detection System) only monitors and alerts on suspicious traffic, while an IPS (Intrusion Prevention System) can actively block the traffic. For WannaCry, an **IPS** would be more effective because it could automatically drop the SMB exploit packets (Port 445) before they infect the host, effectively stopping the worm's spread in real-time.

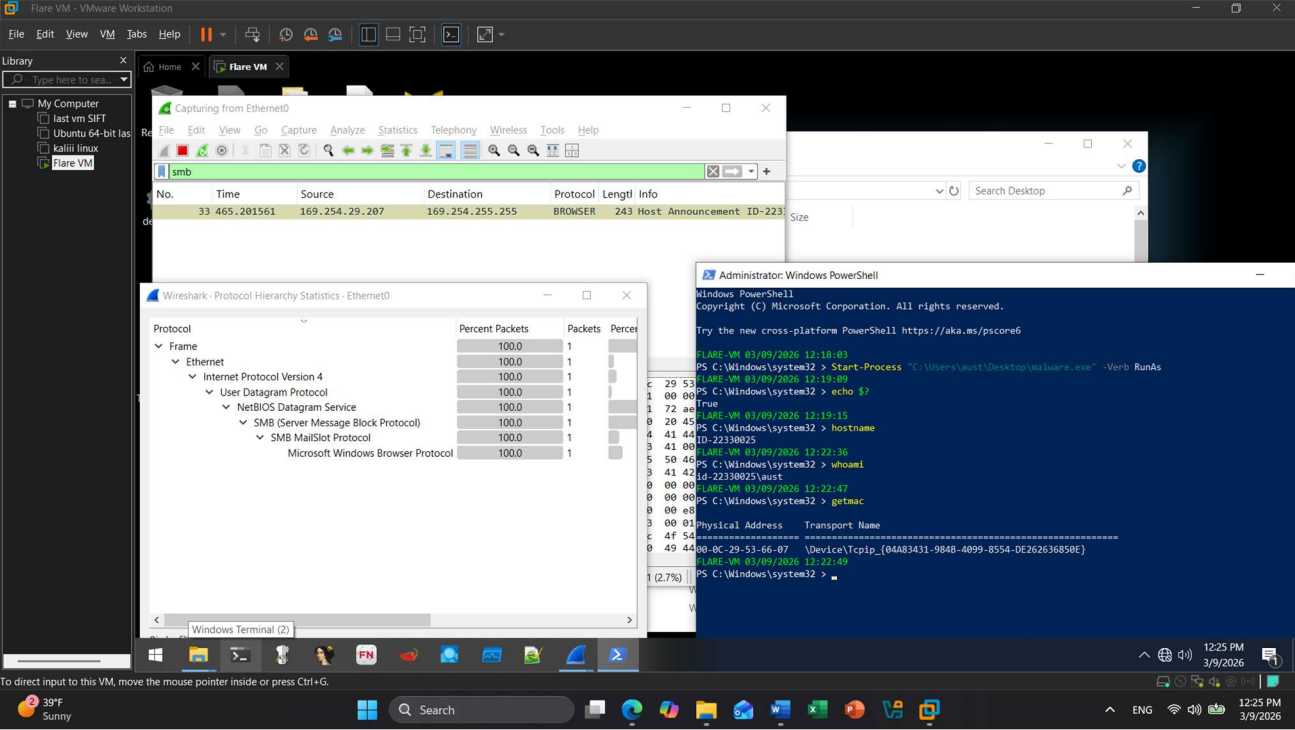
8. Looking at the traffic you captured, describe one sign of anomalous behavior that a network analyst would immediately flag as suspicious (even without knowing it was malware). What filter or statistic in Wireshark revealed it?

An analyst would flag the massive spike in outbound TCP SYN packets to Port 445 across a wide range of IP addresses. This is not normal behavior for a workstation and indicates an automated scanning or exploitation attempt. This was revealed in Wireshark using the `tcp.flags.syn == 1 && tcp.flags.ack == 0` filter and the Conversations statistics window.

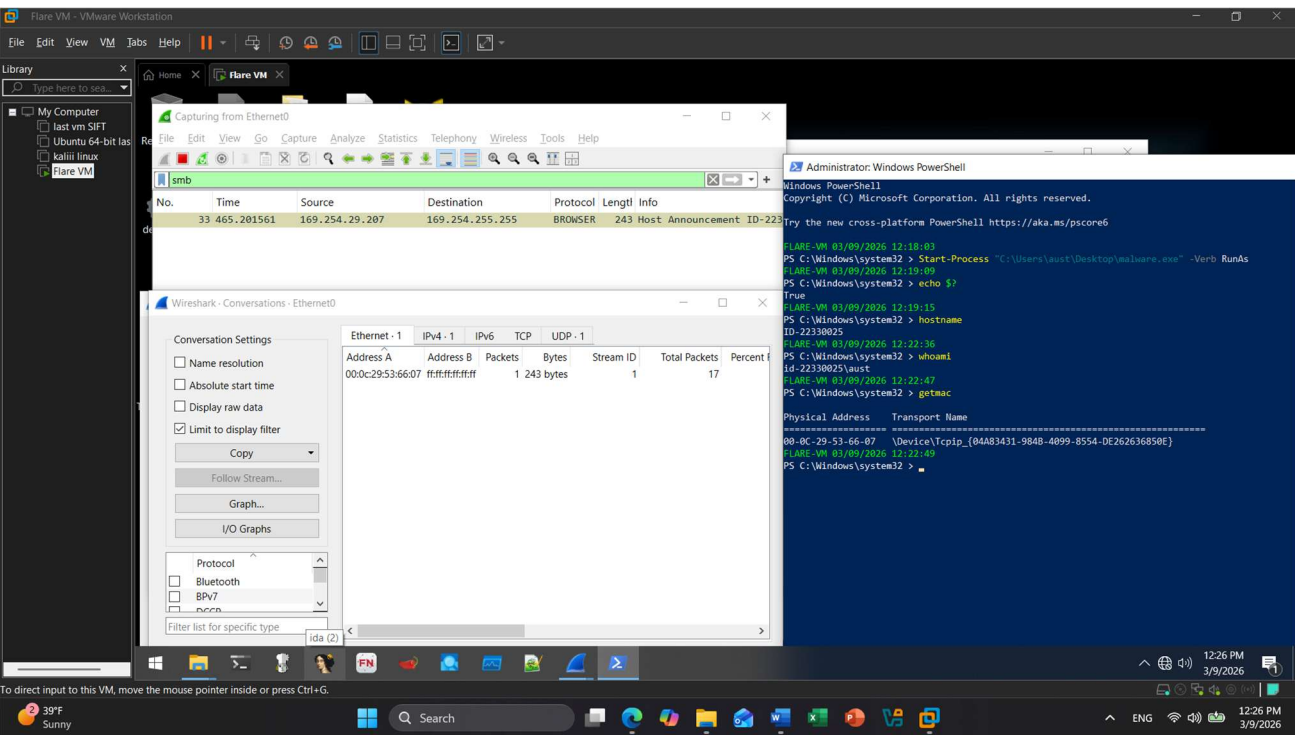
SS-23:



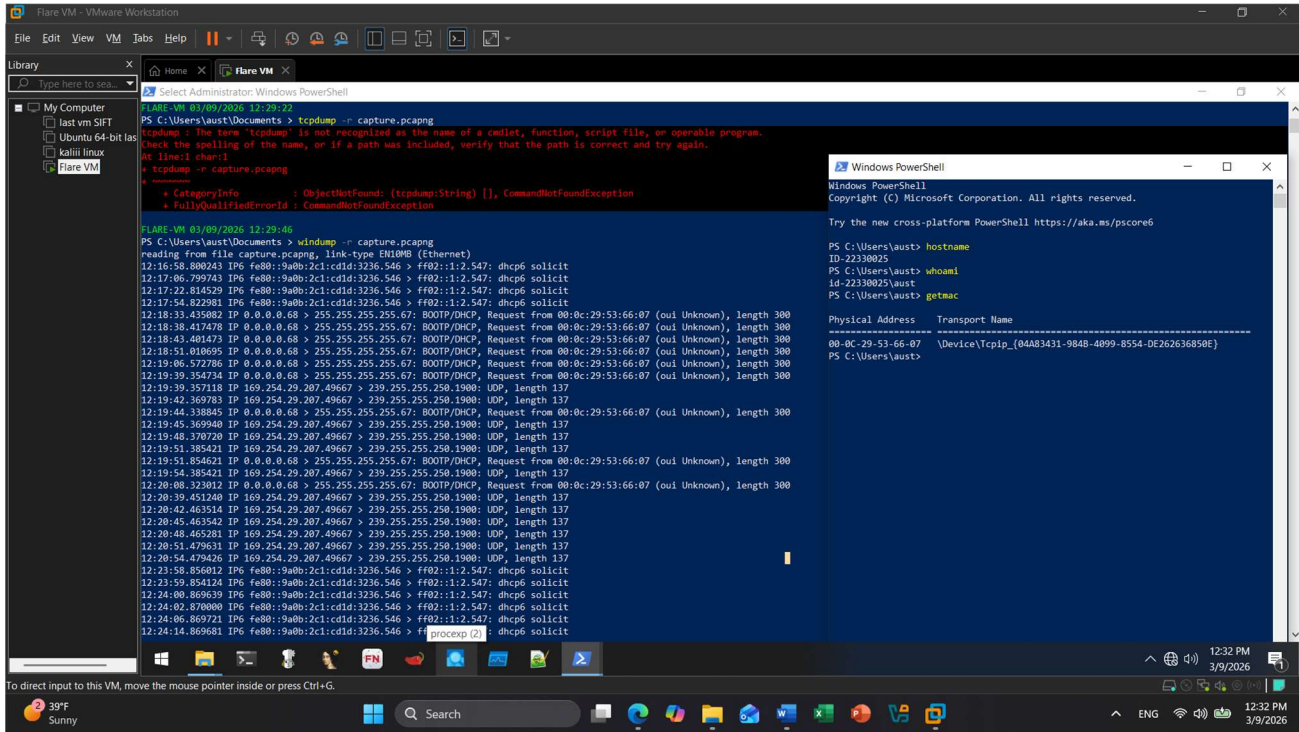
SS-24:



SS-25:



SS-26:



Part C – Required Screenshots

ID	Screenshot Label	What Must Be Visible in the Screenshot
SS-21	Wireshark – DNS Filter	Wireshark with dns display filter applied, showing DNS queries made by the malware. Domain names must be visible.
SS-22	Wireshark – SMB Traffic	Wireshark with smb or smb2 display filter applied, showing SMB packets. At least one packet must be selected with details visible in the lower pane.
SS-23	Wireshark – SYN Scan / Beacon	Wireshark with tcp.flags.syn==1 && tcp.flags.ack==0 filter applied, showing outbound SYN packets. Destination IPs and ports must be visible.
SS-24	Wireshark – Protocol Hierarchy	Wireshark Statistics → Protocol Hierarchy window showing the full protocol breakdown from the capture.
SS-25	Wireshark – Conversations	Wireshark Statistics → Conversations window with TCP tab selected, showing connections sorted by bytes or packets.
SS-26	tcpdump – Running in Terminal	Terminal showing tcpdump running live (or replaying the .pcap) with at least one capture filter applied. Command and output must be visible.

Submission Checklist

Before submitting, verify every item below. Incomplete submissions will receive a 0 on the missing component without the possibility of resubmission.

✓	Checklist Item
✓	GitHub repo named correctly: cybersec-assignment-[StudentID]
✓	README.md contains your name, student ID, malware name, SHA-256 hash, and description
✓	Completed written report (this document) saved as PDF and uploaded to report/
✓	All screenshots uploaded to correct folders with correct naming (SS-XX_label.png)
✓	strings_output.txt uploaded to part-a-static/
✓	procmon_log.csv uploaded to part-b-dynamic/
✓	capture.pcap uploaded to part-b-dynamic/
✓	procdot_graph.png uploaded to part-b-dynamic/
✓	All tables in the report (A.1, A.2, A.3, B.1, B.2, C.1, C.2, tcpdump table) are filled in
✓	All 8 short-answer questions are answered (Q1–Q8)
✓	Both IDS rules are written in the report
✓	VM was reverted to pre-execution snapshot after analysis

Academic Integrity Note

All work submitted must be your own. Screenshots must be taken from your own FLARE-VM instance — screenshots copied from online sources or peers will be detected and considered academic dishonesty. Analysis answers must reflect your own observations from your specific execution environment. Collaboration in the form of discussing concepts is permitted; sharing screenshots, reports, or logs is not.